

## Tema 2: Piles i cues

### 2.1 Convencions de C++

Durant el curs utilitzarem algunes bones pràctiques per escriure codi segur i llegible. Aquestes convencions se sumen als conceptes ja vistos anteriorment (`inline`, `static` o `template`).

- **Nom d'atributs privats** (`_`): Els atributs privats acaben amb guió baix per diferenciar-los intuïtivament dels paràmetres de fora de la classe. Ex: `valor_`.
- **Llista d'inicialitzadors** (`:`): S'utilitza en el `cpp` per construir directament els atributs de la classe abans d'entrar a les claus del constructor, millorant el rendiment respecte assignar-ho a posteriori.
- **Precondicions** (`assert`): Atura l'execució del programa immediatament si una condició és falsa, evitant propagar errors silenciosos i alertant al programador d'irregularitats d'ús. Necessita `#include <cassert>`.

```
“cpp [Caixa.cpp] #include “Caixa.hh” #include
```

```
// Ús de la llista d'inicialitzadors (:) i assert Caixa::Caixa(int valor_inicial) : valor_(valor_inicial) { as-  
sert(valor_inicial >= 0); // Atura el programa si el valor és negatiu }
```

```
*[Simulació interactiva disponible a la web]*
```

```
---
```

```
## 2.2 La pila (`stack`)
```

Estructura de dades lineal amb política **LIFO** (Last In, First Out): l'últim element a entrar és el primer a sortir.

```
### Implementació interna
```

A nivell intern encapsula un `std::vector` i restringeix les operacions exclusivament al seu extrem final.

```
**Exemple simplificat d'interfície <code>Stack.hh</code> pròpia**
```

```
```cpp
```

```
#ifndef STACK_HH
```

```
#define STACK_HH
```

```
#include <vector>
```

```
#include <cassert>
```

```
template <typename T>
```

```
class Stack {
```

```
    std::vector<T> elems_;
```

```

public:
    void push(const T& x) { elems_.push_back(x); }
    void pop() {
        assert(elems_.size() > 0);
        elems_.pop_back();
    }
    const T& top() const {
        assert(elems_.size() > 0);
        return elems_.back();
    }
    bool empty() const { return elems_.empty(); }
    int size() const { return elems_.size(); }
};
#endif

```

### Ús de <stack> de C++

La llibreria de C++ proporciona la classe ja fabricada `std::stack`. Totes aquestes operacions tenen cost constant  $\mathcal{O}(1)$ :

- **push(x)**: Afegeix el valor `x` per damunt (al cim).
- **top()**: Consulta (retorna) l'element del cim sense esborrar-lo.
- **pop()**: Elimina l'element superior. **Només l'elimina, no el retorna.**

```

#include <stack>
using namespace std;

stack<int> S;
S.push(10);           // S: [10]
S.push(20);           // S: [10, 20] <- Cim
int x = S.top();      // Ens guardem x=20
S.pop();              // Extreu el 20. S: [10] <- Cim

```

[Simulació interactiva disponible a la web]

---

## 2.3 La cua (queue)

Estructura de dades lineal amb política **FIFO (First In, First Out)**: el primer a entrar és el primer a sortir. Es tracta d'una cua convencional (s'arriba pel darrere, s'extreu pel davant).

### Implementació interna: L'eficiència

A diferència de la pila, esborrar sistemàticament al començament d'un `std::vector` té un elevat cost  $\mathcal{O}(n)$ , perquè cal desplaçar obligatòriament la resta d'elements per omplir el forat de memòria. Per això, la cua no utilitza un vector de base sinó generalment un `std::deque` (Double Ended Queue), un suport optimitzat que permet tant insercions per la dreta com extraccions per l'esquerra constantment  $\mathcal{O}(1)$ .

### Exemple simplificat d'interfície Queue.hh pròpia usant deque

```

#ifndef QUEUE_HH
#define QUEUE_HH
#include <deque>
#include <cassert>

template <typename T>
class Queue {

```

```

    std::deque<T> elems_;

public:
    void push(const T& x) { elems_.push_back(x); }
    void pop() {
        assert(elems_.size() > 0);
        elems_.pop_front();
    }
    const T& front() const {
        assert(elems_.size() > 0);
        return elems_.front();
    }
    bool empty() const { return elems_.empty(); }
    int size() const { return elems_.size(); }
};
#endif

```

### Ús de <queue> de C++

Proporcionat mitjançant `std::queue`. Mètodes  $\mathcal{O}(1)$ :

- `push(x)`: Afegeix `x` al final (ingressant fila).
- `front()`: Consulta l'element del davant (el primer disposat per sortir).
- `pop()`: Elimina i allibera l'element del davant (sense retornar-lo).

```

#include <queue>
using namespace std;

queue<int> Q;
Q.push(10);           // Q: davant[10]darrere
Q.push(20);           // Q: davant[10, 20]darrere.
int x = Q.front();    // x=10, qui era primer a la fila.
Q.pop();              // Extreu element davanter. Q: davant[20]darrere.

```